Chapter 11: Indexing and Hashing

- Basic Concepts
- Ordered Indices
- B+-Tree Index Files
- B-Tree Index Files

Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
  - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices:** search keys are stored in sorted order
  - **Hash indices:** search keys are distributed uniformly across "buckets" using a "hash function".

Index Evaluation Metrics

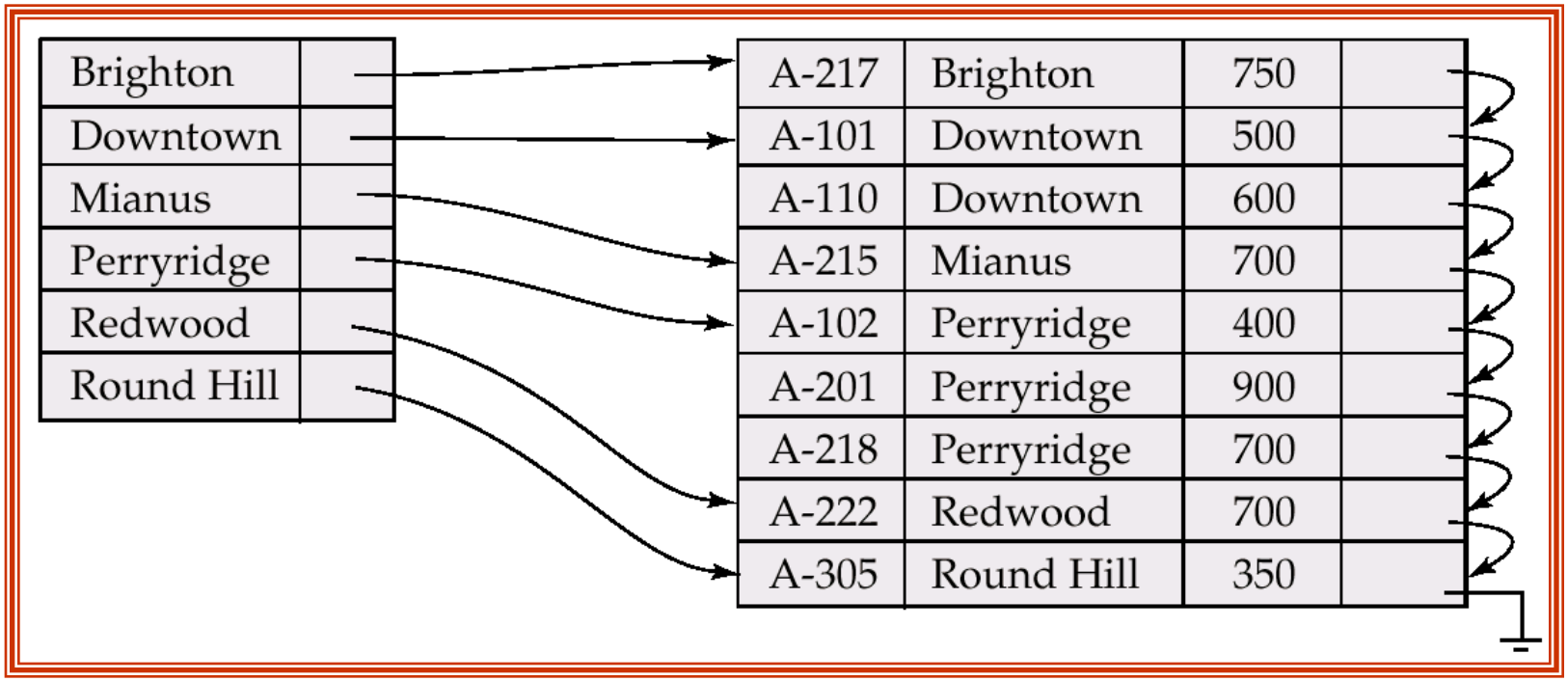
- Access types supported efficiently. E.g.,
  - records with a specified value in the attribute
  - or records with an attribute value falling in a specified range of values.
- Access time
- Insertion time
- Deletion time
- Space overhead

Ordered Indices

- In an **ordered index**, index entries are stored sorted on the search key value. E.g., author catalog in library.
- **Primary index/ clustering index** : in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
  - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index/ non-clustering index** : an index whose search key specifies an order different from the sequential order of the file.
- Index-sequential file: ordered sequential file with a primary index.

Dense Index Files

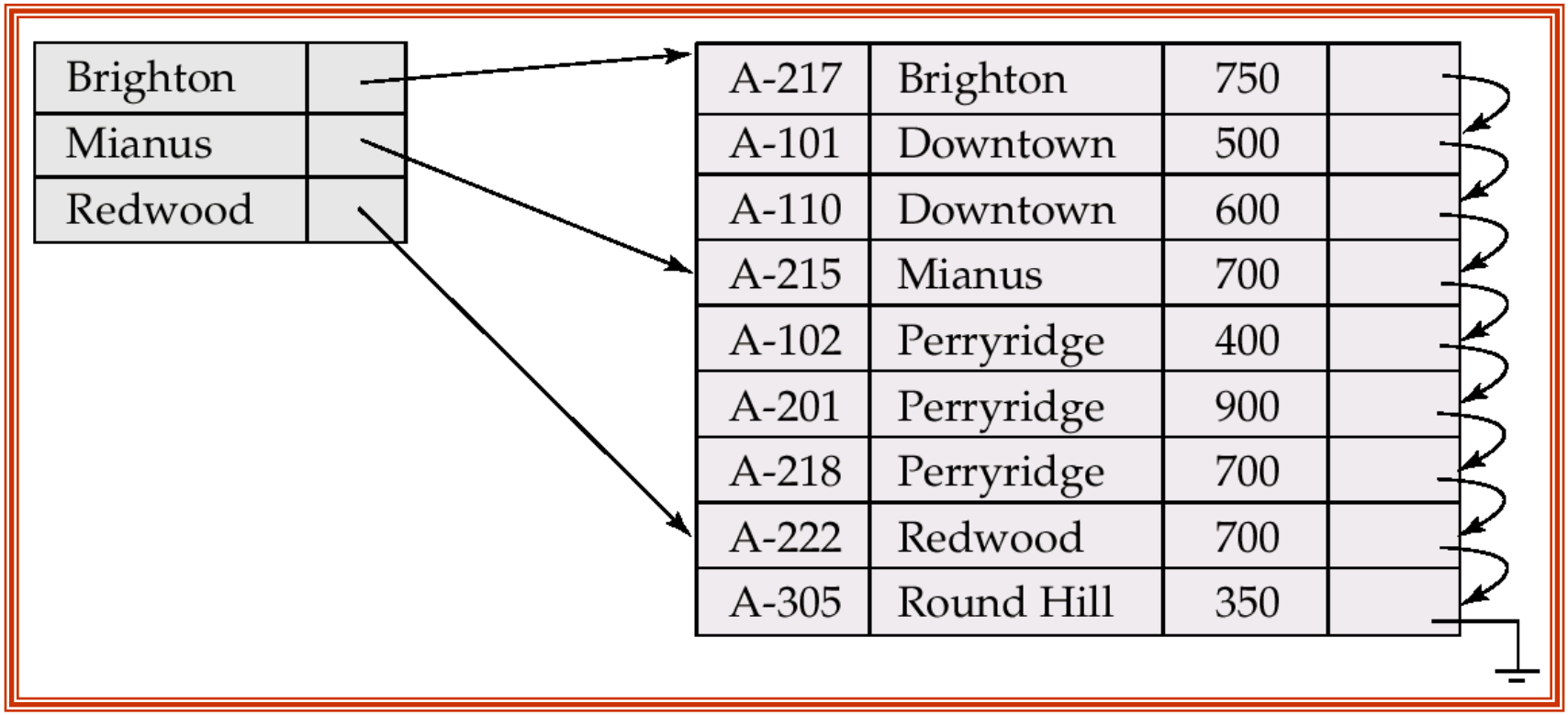
- Dense index — Index record appears for every search-key value in the file.



Sparse Index Files

- Sparse Index: contains index records for only some search-key values.
  - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value *K* we:
  - Find index record with largest search-key value < *K*
  - Search file sequentially starting at the record to which the index record points
- Less space and less maintenance overhead for insertions and deletions.
- Generally slower than dense index for locating records.
- Good tradeoff: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.

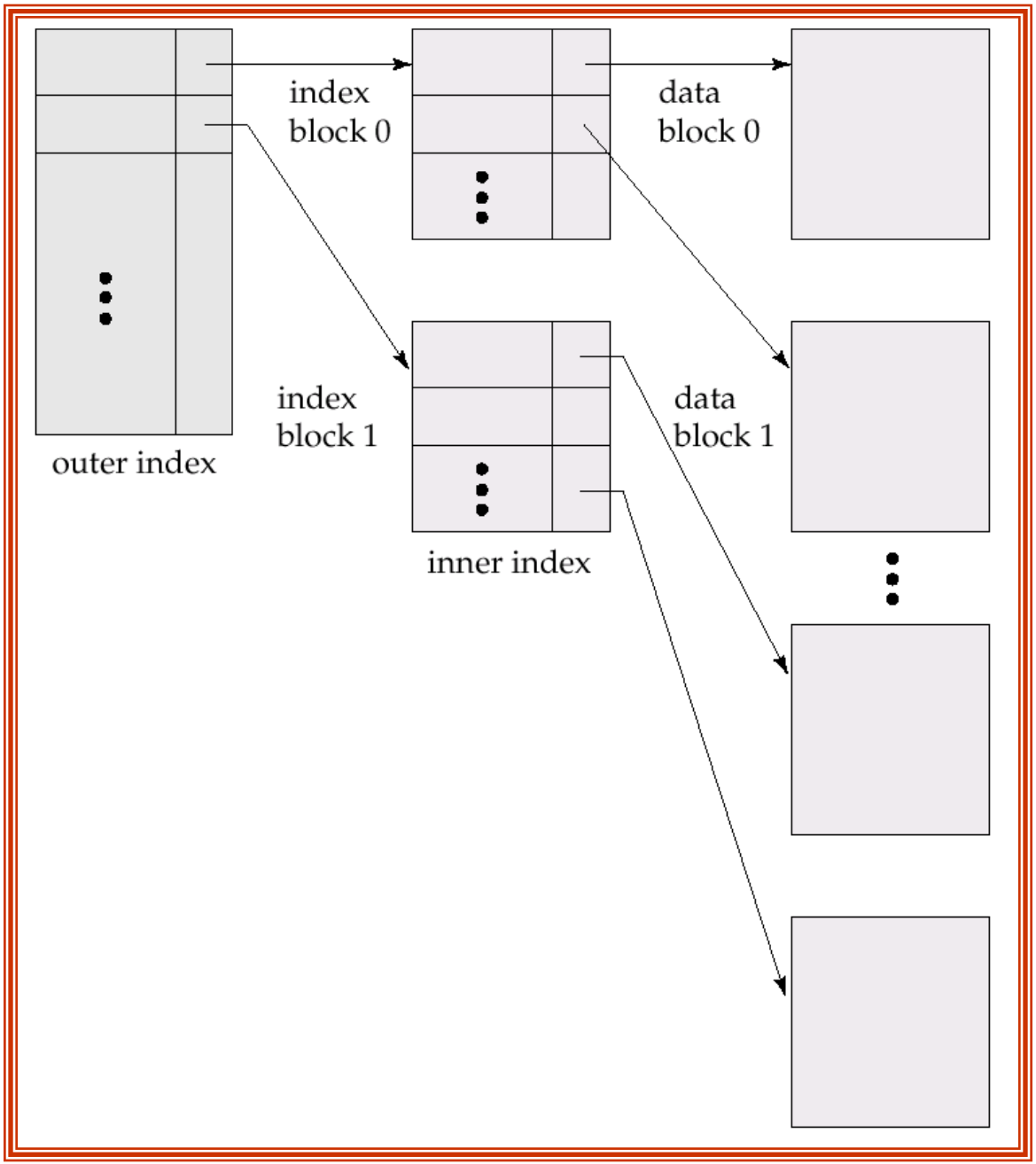
Example of Sparse Index Files



Multilevel Index

- If primary index does not fit in memory, access becomes expensive.
- To reduce number of disk accesses to index records, treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - outer index – a sparse index of primary index
  - inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

Multilevel Index (Cont.)



B Trees: Properties

- No node has more than M children.
- Every node except for the root and the terminal node has at least [M/2]
- The root, unless the tree only has one node, has at least two children.
- All terminal nodes appear on the same level, that is, are the same distance from the root.
- A non-terminal node with k children contain K-1 record and a terminal node contains at least [M/2]-1 and at most [M-1] records.

B Trees: Example

B* Trees: Properties

- No node apart from the root has more than M children.
- Every node except for the root and the terminal node has at least [(2M-2)/3]+1 children
- The root, unless the tree only has one node, has at least two children and at most 2[(2M-2)/3]+1 children
- All terminal nodes appear on the same level, that is, are the same distance from the root.
- A non-terminal node with k children contain K-1 record and a terminal node contains at least [(2M-2)/3] and at most [M-1] records.

B+ Trees: Properties

- The root node has 0, 2 or [M/2] through M children
- All nodes except the root and the terminal node have at least [M/2] children and not more than M children
- All terminal nodes appear on the same level, that is, they are the same distance from the root
- A non-terminal node with k children contain K-1 keys
- Terminal nodes represent the sequence set of the data file and are linked together

Types of indexing (Structural point of view)

- Dynamic Index
- Static Index

Practice Problem

- B Tree: Insert, Delete

End of Chapter